

34 — Deployment & DevOps Specification

Version 1.0

Product-Level Engineering Specification

Core deployment rule: Every production change must be reproducible, reviewable, observable, reversible where practical, and promoted through controlled environments.

Document	Value
System	CloudShield AI
Specification	34 — Deployment & DevOps
Version	1.0
Scope	Product-wide source control, CI/CD, build, release, deployment, configuration, migration, rollback, and operational DevOps
Primary Cloud	AWS for MVP
Status	Engineering Baseline
Depends On	00–33

Document Map

- 1. Purpose and Scope
- 2. DevOps Objectives and Principles
- 3. Deployment Operating Model
- 4. Environment Strategy
- 5. Repository and Source-Control Model
- 6. Branching and Change Management
- 7. Build System
- 8. Artifact Management
- 9. CI Pipeline
- 10. Continuous Integration Quality Gates
- 11. Continuous Delivery Pipeline
- 12. Deployment Strategies
- 13. Infrastructure Delivery
- 14. Application Configuration and Secrets
- 15. Database Schema and Migration Delivery
- 16. Application and Worker Deployment
- 17. DAIL Deployment and Isolation
- 18. AI / LLM Deployment
- 19. Cloud Integration and Provider Adapters
- 20. External Integration Deployment
- 21. Security in the Software Supply Chain
- 22. Testing Strategy in CI/CD
- 23. Release Verification
- 24. Promotion and Governance
- 25. Rollback and Recovery

- 26. Deployment Failure and Reconciliation
- 27. Observability and Release Telemetry
- 28. Feature Flags and Controlled Rollouts
- 29. Versioning and Compatibility
- 30. Release Management
- 31. Change Approvals and Audit
- 32. Access Control and Developer Identity
- 33. Dev Environment and Local Development
- 34. Performance and Reliability Engineering
- 35. Operational Readiness
- 36. Incident and Hotfix Workflow
- 37. Disaster Recovery Integration
- 38. Environment-Specific Policies
- 39. MVP Deployment Baseline
- 40. Implementation Sequence
- 41. Security and Functional Acceptance Criteria
- 42. Traceability
- 43. Engineering Decisions
- 44. Definition of Done
- 45. Glossary

1. Purpose and Scope

This specification defines the engineering practices and technical mechanisms used to build, test, package, release, deploy, operate, and evolve CloudShield AI. It covers application code, workers, infrastructure, database changes, configuration, security controls, DAIL, AI/LLM integration, cloud adapters, external integrations, and release governance.

The objective is to make deployment a controlled continuation of the product lifecycle rather than a separate operational activity. Build artifacts, infrastructure, configuration, migrations, verification results, approvals, and production outcomes must remain traceable.

Deployment changes implementation state; they do not by themselves establish product TRUSTED state.

1.1 In Scope

- Git/source control, code review, branching, CI, artifact creation, release pipelines, deployment orchestration, and rollback.
- Development, test, staging, and production environments.
- AWS/Terraform infrastructure delivery and infrastructure drift controls.
- Application/API, asynchronous workers, DAIL, AI/LLM, cloud adapters, and external integration delivery.
- Database schema migration, configuration, feature flags, secrets, compatibility, and versioning.
- Automated testing, security scanning, release verification, deployment telemetry, and audit.
- Hotfixes, emergency changes, disaster-recovery deployment, and operational readiness.

1.2 Out of Scope

- Replacing the AWS/Terraform architecture and module details in Specification 27.
- Replacing the product security operations model in Specification 32 or disaster recovery model in Specification 33.
- Defining a specific commercial CI/CD vendor as a mandatory dependency.
- Allowing CI/CD automation to bypass application authorization, DAIL verification, governance, or promotion controls.

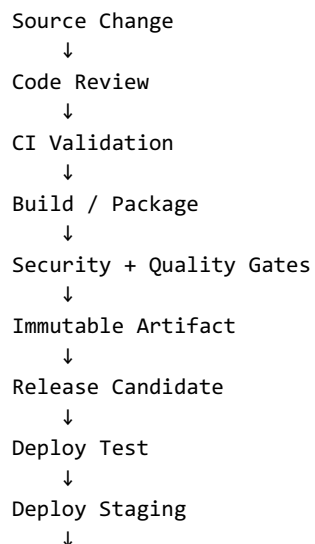
2. DevOps Objectives and Principles

Objective	Requirement	Success Condition
Reproducibility	Same source/configuration produces traceable equivalent artifacts.	Build can be reproduced or independently verified.
Safety	Changes pass automated and human controls before production.	Unsafe changes are blocked.
Traceability	Code, artifact, infrastructure, migration, and deployment are linked.	Production state can be mapped to source and release.
Speed	Automate routine validation and deployment work.	Short feedback loops without reducing control quality.
Reliability	Deployments avoid unnecessary downtime and state corruption.	Failed changes are detected and recoverable.
Security	Protect source, build, artifacts, credentials, and runtime identities.	Supply-chain controls remain intact.
Auditability	Record who/what/when/why for consequential changes.	Release history is reconstructable.
Reversibility	Provide rollback or forward-fix paths.	Failed deployments can return to a safe operating state.

2.1 Core Principles

- Everything-as-code where practical: application, infrastructure, deployment configuration, policy, and pipeline definitions.
- Immutable artifacts: build once, promote the same artifact; do not rebuild separately for production.
- Environment separation: credentials, data, permissions, and deployment targets are isolated.
- Least privilege: CI/CD identities receive only required permissions.
- Progressive exposure: production changes should be gradual when risk warrants it.
- Fail closed: failed security, verification, compatibility, or policy gates stop progression.
- Observability-first: deployment health is measured immediately after release.
- Rollback-aware: every material deployment identifies its rollback or forward-fix strategy.
- Human governance for high-impact changes remains separate from automated execution.

3. Deployment Operating Model



```

Release Verification
↓
Approval / Policy
↓
Production Deployment
↓
Post-Deploy Verification
↓
Observe / Promote / Roll Back

```

3.1 Deployment Planes

Plane	Purpose	Examples
Source Plane	Authoritative engineering changes.	Git repository, pull request, signed commit.
Build Plane	Compile/package/test and produce artifacts.	CI runners, dependency resolution, image build.
Artifact Plane	Store immutable release artifacts.	Container registry, package store, provenance metadata.
Infrastructure Plane	Provision/change runtime resources.	Terraform, AWS APIs.
Deployment Plane	Place approved artifacts/configuration into environments.	Deployment controller, orchestration.
Governance Plane	Authorize progression.	Review, policy checks, DAIL where applicable.
Runtime Plane	Execute deployed workloads.	API, workers, database, queues.
Evidence Plane	Record release proof.	Deployment events, test results, approvals, verification.

4. Environment Strategy

Environment	Purpose	Deployment Policy
Local / Dev	Fast developer feedback and feature development.	No production credentials; synthetic/test data; automated lint/test.
Test	Automated integration/security validation.	Ephemeral or controlled; destructive testing permitted within scope.
Staging	Production-like release validation.	Release candidate only; production-like configuration and security controls.
Production	Customer-facing service.	Approved immutable artifacts, controlled rollout, monitoring and rollback readiness.

- Environment-specific configuration is externalized and versioned where safe.
- Promotion moves the same immutable artifact across environments whenever practical.
- Production data must not be copied into lower environments unless explicitly authorized and protected.
- Developer access to production is exceptional and least-privileged.
- Environment differences must be documented rather than hidden in deployment scripts.

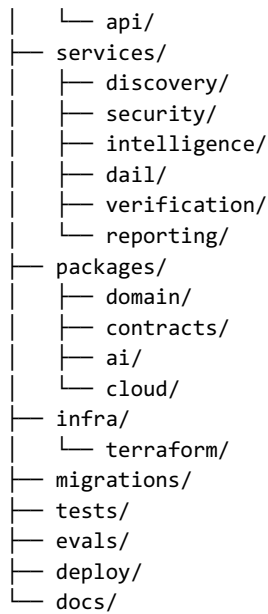
5. Repository and Source-Control Model

5.1 Recommended Repository Structure

```

cloudshield-ai/
├─ apps/
└─ web/

```



- Exact repository decomposition may evolve; logical ownership and dependency boundaries must remain explicit.
- Shared domain contracts must not import provider-specific SDKs directly.
- DAIL code is isolated as a bounded subsystem with explicit contracts.
- Pipeline definitions and infrastructure changes are reviewed like application code.

6. Branching and Change Management

- Use a protected main branch as the integration source for releasable code.
- Short-lived feature branches or equivalent review-based workflows are preferred.
- Every production-impacting change requires review and automated validation.
- Security-sensitive changes require appropriate security-owner review.
- Database migrations, IAM changes, Terraform changes, and DAIL lifecycle changes receive explicit risk classification.
- Emergency hotfixes follow a controlled exception path and are reconciled back into the normal source branch.

Change Type	Minimum Control
Normal application change	Code review + CI + tests + artifact scan.
Security-sensitive change	Normal controls + security review/risk assessment.
Infrastructure/IAM change	Code review + Terraform plan + security policy checks + deployment verification.
Database migration	Migration review + compatibility tests + backup/recovery plan.
DAIL verification/promotion change	Code review + DAIL-specific tests + lifecycle safety review.
AI/LLM safety change	Code review + evaluation/regression suite + safety review.
Emergency hotfix	Incident authorization + expedited validation + post-change review.

7. Build System

- Builds must be deterministic or document controlled sources of nondeterminism.
- Pin important dependency versions and record lockfiles.
- Use reproducible base images and build environments where practical.
- Build metadata includes source revision, build timestamp, dependency manifest, toolchain version, and artifact digest.
- Build systems must not require production credentials.

- Network access from builds should be restricted to required dependency/artifact sources.
- Failed builds do not produce releasable artifacts.

7.1 Artifact Identity

Field	Requirement
artifact_id	Stable logical artifact identifier.
version	Human-readable release version.
digest	Cryptographic content digest for immutable identity.
source_revision	Exact Git commit/revision.
build_id	Unique CI build identifier.
dependencies	Resolved dependency set/lockfile reference.
provenance	Build system and source metadata.
security_results	Required scan/test result references.

8. Artifact Management

- Container images/packages are immutable after publication.
- Use content digests rather than mutable tags as the authoritative production reference.
- Artifacts are promoted across environments rather than rebuilt for each environment.
- Retain release artifacts for the period required for rollback, audit, and recovery.
- Scan dependencies and images before release.
- Restrict artifact publishing and deletion permissions.
- Production deployment records include the exact artifact digest.

A mutable tag is a convenience pointer; the immutable digest is the release identity.

9. CI Pipeline

```

Pull Request
  ↓
Checkout + Dependency Restore
  ↓
Lint / Format / Type Checks
  ↓
Unit Tests
  ↓
Static Analysis
  ↓
Security / Secret / Dependency Scans
  ↓
Integration / Contract Tests
  ↓
Build
  ↓
Artifact Scan
  ↓
Publish Immutable Artifact
  ↓
Attach Provenance + Results

```

- CI must fail on configured blocking security or quality gates.
- Test results are retained and associated with the source revision.

- CI does not deploy to production directly from an unreviewed branch.
- Pipeline failures are visible through deployment/engineering observability.

10. Continuous Integration Quality Gates

Gate	Blocking Condition
Formatting/Lint	Required checks fail beyond configured threshold.
Compilation/Type	Build or type validation fails.
Unit Tests	Required tests fail.
Contract Tests	API/event/domain contract compatibility fails.
Security Scan	Blocking vulnerability/secrets/policy threshold is exceeded.
Dependency Policy	Disallowed dependency/license/security condition is detected.
Container/Image	Artifact fails required security baseline.
DAIL Tests	DAIL lifecycle/invariant/verification tests fail.
AI Evaluation	Required safety/quality regression threshold fails.
Infrastructure	Terraform validation/plan/policy checks fail.
Provenance	Artifact cannot be tied to source/build metadata.

11. Continuous Delivery Pipeline

```

Immutable Artifact
  ↓
Deploy Test
  ↓
Automated Verification
  ↓
Deploy Staging
  ↓
Release Candidate Validation
  ↓
Approval / Policy
  ↓
Production Canary / Controlled Rollout
  ↓
Post-Deploy Verification
  ↓
Complete / Roll Back / Forward Fix

```

- Deployment state is recorded independently from application runtime logs.
- Each stage has explicit entry/exit criteria.
- Production progression is blocked when critical verification fails.
- Deployment pipelines must be idempotent where practical.

12. Deployment Strategies

Strategy	Use	Risk Control
Rolling	Default for stateless application/worker capacity.	Health checks + bounded batch size.
Canary	High-risk or high-impact changes.	Small traffic exposure + automated metrics.
Blue/Green	Changes requiring rapid traffic switch.	Independent environments + validation.

Recreate	Only when necessary for incompatible workloads.	Maintenance window + rollback/recovery plan.
Migration-first	Backward-compatible database transition.	Expand/contract pattern.
Feature Flag	Decouple deployment from feature activation.	Flag ownership, expiry, audit.

Deployment strategy must be selected by risk and compatibility, not convenience.

13. Infrastructure Delivery

- Terraform is the primary infrastructure delivery mechanism for MVP.
- Terraform plans are reviewed before sensitive infrastructure changes.
- Production applies use controlled identities and protected state backends.
- Terraform state is not stored in application source repositories.
- Infrastructure modules are versioned and tested.
- Security policies evaluate IAM, networking, encryption, logging, public exposure, and required tags.
- Drift is surfaced as explicit operational/security state.

```

Terraform Source
  ↓
fmt / validate
  ↓
Static Policy Checks
  ↓
Plan
  ↓
Review / Approval
  ↓
Apply
  ↓
AWS Verification
  ↓
Drift / Observability

```

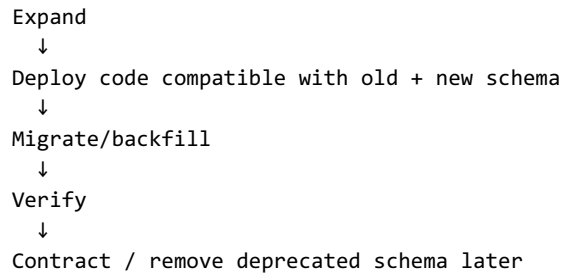
14. Application Configuration and Secrets

- Separate deployable artifact from environment configuration.
- Non-secret configuration may be version-controlled where appropriate.
- Secrets use AWS Secrets Manager or equivalent managed secret systems.
- CI/CD does not print secret values into logs.
- Secret access is granted to runtime identities, not broadly to developers or build agents.
- Configuration changes are versioned/audited when security or behavior is material.
- Secret rotation must be compatible with application rollout and recovery.

15. Database Schema and Migration Delivery

- Schema migrations are versioned and applied in a controlled sequence.
- Prefer backward-compatible expand/contract migrations for zero/minimal downtime.
- Application releases must account for schema compatibility during rolling deployments.
- Destructive migrations require explicit backup/recovery consideration.
- Migration status is recorded durably.
- Failed migrations stop progression and trigger a defined repair or rollback strategy.

- Rollback of application code does not imply database rollback; forward-compatible migration design is preferred.

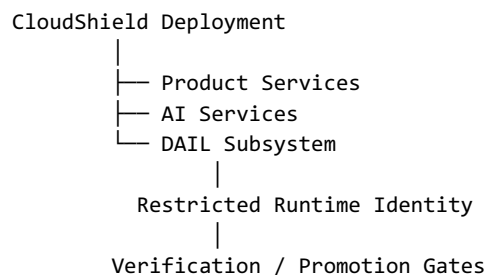


16. Application and Worker Deployment

- API and UI releases are versioned and traceable to exact artifacts.
- Workers use explicit queue/job compatibility versions.
- Worker concurrency is controlled during rollout.
- Long-running jobs must survive deployment or be safely requeued.
- Deployment termination must honor graceful shutdown where supported.
- Readiness gates prevent traffic before required dependencies are available.
- Worker startup must validate database, queue, secrets, and authorization dependencies.

17. DAIL Deployment and Isolation

- DAIL is deployed as a bounded subsystem within CloudShield AI, not as the entire product.
- DAIL interfaces use explicit versioned contracts.
- DAIL deployment changes require lifecycle and invariant regression tests.
- Verification and promotion capabilities receive stricter runtime permissions than ordinary application/AI workloads.
- Deployment of DAIL code must not grant the deployment pipeline unrestricted promotion authority.
- Changes affecting candidate immutability, verification, evidence, dependency impact, or promotion require security/lifecycle review.
- DAIL deployment health must be independently observable.



18. AI / LLM Deployment

- AI gateway and provider adapters are deployed independently from provider-specific model credentials.
- Model/provider configuration is versioned and auditable.
- Prompt templates, output schemas, safety policies, and routing policies are treated as deployable artifacts.
- AI releases require evaluation/regression checks appropriate to the affected task.
- High-impact model/routing changes require safety review.
- Changing model provider does not remove output validation, tool restrictions, DAIL verification, or governance.
- Local/open-weight model adapters can be introduced later without changing the domain layer.

19. Cloud Integration and Provider Adapters

- Provider adapters are versioned independently from the provider-neutral domain contracts.
- AWS SDK/client versions are pinned and tested.
- Cloud integration changes require mock tests plus controlled AWS integration tests.
- Read-path changes and mutation-path changes receive separate risk classification.
- Mutation-capable deployment identities are tightly scoped and not available to generic CI jobs.
- Provider API behavior changes are monitored and reconciled through authoritative observations.

20. External Integration Deployment

- OAuth scopes, webhook verification, connector permissions, and integration configuration are deployed through controlled configuration.
- Integration adapters must support idempotency and safe retry semantics.
- External-system changes cannot bypass CloudShield lifecycle controls.
- Webhook consumers are backward compatible during rolling deployment where possible.
- Connector secrets are isolated from unrelated workloads.
- External integration failure must not block critical product recovery unless explicitly required.

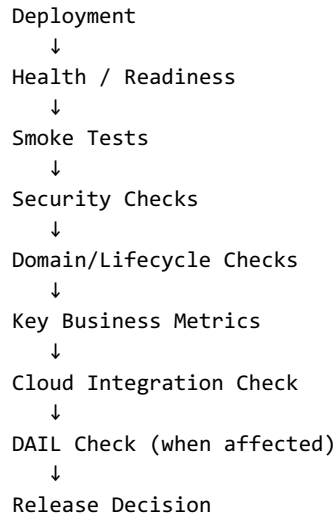
21. Security in the Software Supply Chain

Control	Requirement
Source Integrity	Protected branches, reviewed changes, appropriate commit/identity controls.
Dependency Security	Pinned/managed dependencies and vulnerability scanning.
Secret Detection	Automated secret scanning before artifact release.
Build Isolation	Least-privileged, controlled CI execution environments.
Artifact Integrity	Immutable digests and provenance.
Image Security	Base-image scanning and patch policy.
IaC Security	Terraform static/policy scanning.
Access Control	Separate developer, CI, deployer, and runtime identities.
Audit	Record pipeline, artifact, approval, and deployment actions.
Recovery	Retain known-good artifacts for rollback/DR.

22. Testing Strategy in CI/CD

- Unit tests for domain logic and lifecycle invariants.
- Integration tests for databases, queues, AWS adapters, and external integration boundaries.
- Contract tests for API, events, AI structured outputs, and DAIL interfaces.
- Security tests for authentication, authorization, tenant isolation, secrets, and injection defenses.
- AI evaluation tests for grounding, structured output, safety, and prompt-injection resistance.
- DAIL tests for candidate/version immutability, dependency impact, verification, evidence, and promotion gates.
- End-to-end tests for golden-path product workflows.
- Failure-injection tests for deployment, dependency outage, stale state, and ambiguous outcomes.

23. Release Verification



- Verification must be based on observed runtime behavior, not only deployment success.
- Production verification checks error rates, latency, job failures, authorization failures, security alerts, and critical workflow health.
- Material verification failure blocks further rollout or initiates rollback/containment.
- Release verification evidence is retained.

24. Promotion and Governance

- Deployment promotion and product-state promotion are separate concepts.
- CI/CD can promote an artifact between environments only when deployment gates succeed.
- Application deployment cannot set product records to VERIFIED, APPROVED, PROMOTED, or TRUSTED.
- Changes that alter infrastructure or cloud state remain subject to the CloudShield lifecycle and governance model.
- High-impact production releases require explicit approval according to policy.
- Every production release records artifact digest, source revision, approver, deployment identity, target environment, and verification result.

Deployment approval answers 'may this software version run here?'; product governance answers 'may this proposed infrastructure state become trusted?'

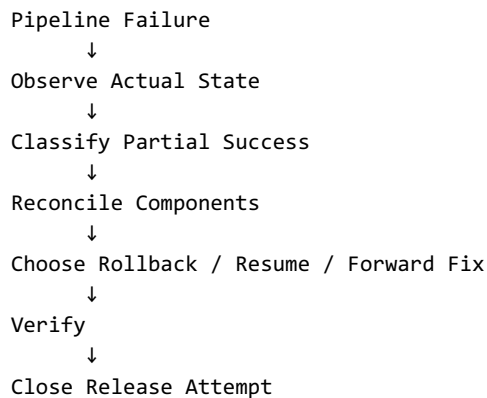
25. Rollback and Recovery

Failure	Preferred Response
Pre-deploy failure	Do not deploy; fix source/artifact.
Deployment health failure	Stop rollout and rollback if safe.
Application regression	Rollback to known-good artifact or forward-fix.
Schema incompatibility	Stop progression; use compatible application or repair migration.
Security regression	Contain/rollback and open security incident as required.
Cloud mutation regression	Stop further mutation, reconcile authoritative state, then remediate.
DAIL integrity regression	Disable affected progression, preserve evidence, re-verify.
Infrastructure failure	Rollback Terraform change or execute controlled recovery.
Data corruption	Freeze unsafe writes and execute DR/database recovery procedure.

- Rollback plans are tested for critical services.
- Rollback does not erase audit or evidence of the failed release.
- Rollback may be impossible or unsafe for irreversible database/cloud mutations; forward recovery is then required.

26. Deployment Failure and Reconciliation

- A deployment that partially succeeds is a state-reconciliation problem, not merely a pipeline error.
- Record which components reached which versions.
- Compare desired deployment state with observed runtime state.
- Check database migration state before retry.
- Check queue/job side effects before replaying workers.
- Check cloud resources before reapplying infrastructure mutations.
- Do not assume a failed pipeline means the deployment had no side effects.



27. Observability and Release Telemetry

- Every deployment has deployment_id, release_id, artifact_digest, source_revision, environment, actor, and timestamp.
- Deployment telemetry integrates with Specification 31's logs, metrics, traces, events, audit, and evidence.
- Track deployment duration, failure rate, rollback rate, post-deploy error rate, and change-related incidents.
- Correlate deployment IDs with application traces, cloud changes, DAIL events, and security incidents.
- Alert on abnormal post-deployment behavior.

28. Feature Flags and Controlled Rollouts

- Feature flags separate code deployment from feature activation.
- Flags are scoped by environment/tenant/feature where required.
- Sensitive flags are authorization-controlled and audited.
- Temporary flags have owners and expiry dates.
- Feature flags must not be used to bypass lifecycle or security gates.
- Rollouts should support gradual exposure and rapid disablement.
- Flag state is configuration, not trusted infrastructure state.

29. Versioning and Compatibility

Artifact/Contract	Versioning Rule
Application	Semantic or equivalent release version + immutable digest.

API	Explicit backward-compatibility and version policy.
Events	Versioned schemas with compatibility rules.
Database	Ordered migration versions.
AI Output Schema	Versioned structured contract.
DAIL Contract	Versioned lifecycle/verification contract.
Terraform Modules	Versioned modules with controlled upgrades.
Policy	Immutable policy versions referenced by decisions.
Detection	Versioned detection IDs/rules.
Prompt/Guardrail	Versioned configuration artifacts.

- Breaking changes require explicit migration strategy.
- Production consumers must not silently receive incompatible contracts.
- Old versions are retired only after dependency inventory and migration verification.

30. Release Management

- Each release has a release ID, scope, owner, artifact list, risk classification, test summary, deployment plan, rollback/forward-fix plan, and verification criteria.
- Release notes distinguish code, infrastructure, configuration, schema, AI, DAIL, and security changes.
- High-risk releases should use smaller rollout batches or scheduled windows where appropriate.
- Release status remains visible during deployment and post-deployment observation.
- Release closure records final outcome and unresolved follow-up.

Risk	Typical Control
Low	Automated CI + standard deployment verification.
Medium	Expanded testing + staging validation + monitored rollout.
High	Security/lifecycle review + canary/controlled rollout + explicit approval.
Critical	Dedicated change plan, senior approval, extensive verification, rollback/recovery readiness.

31. Change Approvals and Audit

- Approvals are tied to an exact release/artifact and expire when material inputs change.
- Approval records include actor, role, timestamp, decision, scope, and policy context.
- Changing the artifact digest invalidates approval.
- Material Terraform, IAM, database, AI safety, and DAIL changes may require specialized approval.
- Emergency approvals are explicitly labeled and require post-change review.
- Audit records cannot be edited to hide failed or reverted releases.

32. Access Control and Developer Identity

Identity	Default Authority
Developer	Source code, local/test environments, pull requests.
CI Identity	Build/test/artifact publishing only; no broad production mutation.
Deploy Identity	Deploy approved artifacts to specified environment.
Terraform Identity	Infrastructure changes within scoped account/environment.

Runtime Identity	Service-specific runtime access only.
DAIL Promotion Identity	Highly restricted promotion capabilities; never generic CI.
Break-Glass Identity	Exceptional, time-bound emergency access with full audit.

CI is not production by default. Build authority, deployment authority, and runtime authority are separate security boundaries.

33. Dev Environment and Local Development

- Local development uses safe test credentials or mocked provider adapters.
- Never require customer production credentials for normal development.
- Provide local database/queue/object-store equivalents where practical.
- Mock LLM/provider adapters support deterministic development and tests.
- AWS integration tests use controlled test accounts/scopes.
- Developer tooling must not silently call customer production resources.
- Local environment configuration is documented and reproducible.

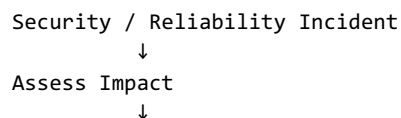
34. Performance and Reliability Engineering

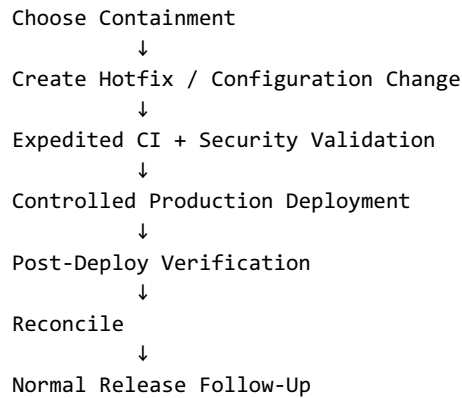
- Measure build duration, test duration, deployment duration, startup time, migration duration, and rollback time.
- Performance regressions are detected before production for defined critical paths.
- Load tests are executed for releases affecting high-volume APIs/workers.
- Deployment concurrency is controlled to protect shared dependencies.
- Capacity changes are tested against AWS quotas and runtime limits.
- Release engineering must account for queue backlogs, database connections, cloud API throttling, and AI provider quotas.

35. Operational Readiness

Readiness Check	Requirement
Health	Readiness/liveness checks are implemented.
Observability	Logs, metrics, traces, audit, and release telemetry available.
Security	Required scans, IAM, secrets, and security controls validated.
Data	Migration/backup/restore strategy documented.
Rollback	Rollback or forward-fix path identified and tested where practical.
Dependencies	AWS, AI, database, queue, and external integrations assessed.
Runbooks	Known failure modes have operational procedures.
Capacity	Expected workload and quotas validated.
Support	Owner/escalation path identified.

36. Incident and Hotfix Workflow





- Hotfixes retain incident linkage and audit evidence.
- Emergency deployment authority is time-bound and scope-limited.
- Hotfixes must be merged/reconciled into the normal source line.
- Security operations and incident command remain involved for security-impacting incidents.

37. Disaster Recovery Integration

- Deployment automation must be usable during disaster recovery.
- Known-good application and infrastructure artifacts are retained according to Specification 33.
- Recovery environments use the same controlled source/artifact process as production.
- Post-restore deployment verifies database/schema compatibility before enabling services.
- DR failover must reconcile in-flight releases and deployment metadata.
- Deployment automation must not assume the primary region is available.

38. Environment-Specific Policies

Environment	Policy
Development	Fast feedback; relaxed scale; no production secrets; broad test instrumentation.
Test	Automated destructive/security tests; ephemeral resources preferred.
Staging	Release-candidate-only; production-like security and configuration.
Production	Protected deployment identities, approvals, immutable artifacts, progressive rollout, enhanced monitoring.
DR/Recovery	Known-good artifacts, controlled identities, recovery evidence, reconciliation before normal mutation.

39. MVP Deployment Baseline

- Git-based protected main branch and pull-request review workflow.
- Automated CI for linting, unit tests, integration/contract tests, security scans, and builds.
- Immutable container/package artifacts stored in a protected registry.
- Artifact digest and source-revision traceability.
- Terraform-based infrastructure deployment with plan/review/apply controls.
- Separate CI, deployment, Terraform, runtime, and DAIL identities.
- Development, test, staging, and production environments.
- Automated staging deployment and controlled production rollout.
- Database migration framework with backward-compatible migration policy.

- Managed secrets and environment configuration.
- Post-deploy smoke, health, security, and lifecycle verification.
- Rollback/forward-fix procedures.
- Deployment telemetry integrated with observability/audit.
- AI/DAIL/cloud-adapter regression gates.
- DR-compatible rebuild and deployment path.

40. Implementation Sequence

1. Create repository structure and ownership boundaries.
2. Establish protected source-control workflow and code-review policy.
3. Build baseline CI pipeline with tests and static/security checks.
4. Create immutable artifact build and registry workflow.
5. Implement Terraform CI plan/policy/apply pipeline.
6. Create environment configuration and managed-secret strategy.
7. Implement database migration framework and compatibility checks.
8. Deploy application/API to development and test environments.
9. Deploy workers/queues and validate graceful lifecycle behavior.
10. Add DAIL-specific deployment and verification gates.
11. Add AI/LLM evaluation gates and provider configuration management.
12. Add AWS/cloud-adapter integration tests.
13. Add staging release pipeline and post-deploy verification.
14. Add production controlled rollout and rollback automation.
15. Integrate deployment telemetry, audit, security operations, and DR.
16. Run deployment-failure, rollback, recovery, and hotfix drills.

41. Security and Functional Acceptance Criteria

ID	Acceptance Criterion
DEVOPS-AC-001	Every production deployment is traceable to an exact source revision and immutable artifact digest.
DEVOPS-AC-002	CI blocks releases on configured critical test/security/provenance failures.
DEVOPS-AC-003	Production deployment identities are separated from generic CI build identities.
DEVOPS-AC-004	Terraform production changes require plan validation and controlled execution.
DEVOPS-AC-005	Production secrets are never embedded in source code, artifacts, or CI logs.
DEVOPS-AC-006	Database migrations are versioned and tested for deployment compatibility.
DEVOPS-AC-007	DAIL deployment changes pass lifecycle/invariant regression tests.
DEVOPS-AC-008	AI/LLM changes affecting safety or high-impact behavior pass required evaluation gates.
DEVOPS-AC-009	Post-deploy verification observes actual runtime health before rollout completion.
DEVOPS-AC-010	Failed/partial deployments trigger reconciliation rather than blind retry.
DEVOPS-AC-011	Rollback or forward-fix strategy exists for critical deployment classes.
DEVOPS-AC-012	Deployment events integrate with logs, metrics, traces, audit, and evidence.
DEVOPS-AC-013	Feature flags and deployment controls cannot bypass security/lifecycle governance.
DEVOPS-AC-014	Emergency hotfixes are fully audited and reconciled into normal source control.
DEVOPS-AC-015	CI/CD cannot directly manufacture VERIFIED, APPROVED, PROMOTED, or TRUSTED product state.

42. Traceability

Upstream Specification	Deployment & DevOps Dependency
00 — Master System	Product control loop, trust model, product boundary.
01 — SRS	Deployment, security, reliability, compatibility, observability requirements.
02 — Architecture	Runtime topology, service boundaries, infrastructure, deployment model.
03 — Functional	Lifecycle workflows, async behavior, failure/recovery semantics.
04 — Product & UX	Release-visible states, operational UX, degraded states.
05 — Domain & Data	Lifecycle records, configuration, evidence, audit, jobs.
06 — State & Lifecycle	State invariants, version binding, reconciliation, trusted-state rules.
07 — Identity & Dependency	Identity, dependency/impact changes affecting deployment verification.
08 — Security & Threat Model	Supply chain, IAM, secrets, attack surface, security controls.
09 — Intelligence & AI Architecture	AI abstraction, routing, deployment-neutral model architecture.
10 — LLM Integration	Provider adapters, model routing, configuration, failure handling.
11 — AI Safety	AI evaluation, safety gates, tool restrictions, prompt-injection defenses.
12–26 — DAIL Package	DAIL deployment, testing, experiment, verification, evidence, DevOps, implementation controls.
27 — AWS & Terraform	AWS infrastructure, Terraform, IAM, secrets, observability, DR.
28 — API	API compatibility, auth, versioning, deployment verification.
29 — Cloud Integration	Provider adapter deployment, AWS testing, mutation safety.
30 — External Integration	Connector/webhook/OAuth deployment and compatibility.
31 — Observability & Logging	Deployment telemetry, release metrics, audit and evidence.
32 — Security Operations	Security gates, detection, incident/hotfix handling.
33 — Disaster Recovery	Known-good artifacts, rebuild, failover, recovery deployment.

43. Engineering Decisions

- Build once, promote immutable artifacts; production is not rebuilt from source independently.
- CI, deployment, Terraform, runtime, and DAIL identities remain separate.
- Deployment governance is distinct from product trusted-state governance.
- Terraform is the infrastructure-as-code baseline for MVP.
- Staging is the primary production-like release validation environment.
- Rolling deployment is the default; canary/blue-green are used when risk or compatibility requires them.
- Backward-compatible database migrations are preferred over destructive rollback.
- DAIL remains a bounded subsystem and receives stricter deployment controls.
- AI model/provider configuration is deployable and versioned, but AI remains non-authoritative.
- Deployment failures are treated as distributed-state reconciliation problems when side effects may have occurred.
- DR must be able to rebuild the product from known-good source, artifacts, infrastructure definitions, and protected data.

44. Definition of Done

- Source-control protections and review workflow are operational.
- CI validates code quality, tests, security, provenance, and artifact integrity.
- Immutable artifacts are published and traceable to source.

- Terraform infrastructure delivery is automated and controlled.
- Environment configuration and secrets are securely managed.
- Database migrations are versioned, tested, and deployment-compatible.
- Application and worker deployment paths are implemented.
- DAIL has explicit deployment boundaries and regression gates.
- AI/LLM changes have appropriate evaluation/safety gates.
- AWS/cloud adapters and external integrations have controlled deployment/testing paths.
- Post-deploy verification and deployment telemetry are operational.
- Rollback/forward-fix and partial-deployment reconciliation are tested.
- Hotfix and DR deployment workflows are documented and exercised.
- Production deployment cannot bypass governance or directly create trusted state.
- Traceability to Specifications 00–33 is maintained.

45. Glossary

Term	Definition
CI	Continuous Integration: automated validation of source changes.
CD	Continuous Delivery/Deployment: controlled delivery of validated artifacts to environments.
Artifact	Immutable deployable package such as a container image or application package.
Artifact Digest	Cryptographic identifier for exact artifact contents.
Release	Defined set of software/configuration/infrastructure changes delivered together.
Deployment	Act of placing an approved release into a target environment.
Promotion	Moving a release artifact or product state through a controlled lifecycle; deployment promotion and product-state promotion are distinct.
Canary	Small controlled production exposure used to validate a release before wider rollout.
Blue/Green	Two independently deployable environments used for controlled traffic switching.
Feature Flag	Runtime configuration controlling feature activation separately from deployment.
Terraform	Infrastructure-as-code tooling used as the MVP infrastructure delivery baseline.
Migration	Versioned database schema/data transformation.
Rollback	Returning runtime software/configuration to a prior known-good state where safe.
Forward Fix	Corrective release applied when rollback is unsafe or insufficient.
DAIL	CloudShield AI subsystem responsible for dependency-aware lifecycle, impact, verification, and related controls.
Trusted State	State earned only after required verification, governance, promotion, and authoritative confirmation.
Provenance	Metadata establishing where an artifact or result came from and how it was produced.
Reconciliation	Process of resolving differences between expected, deployed, and authoritative observed state.

Final deployment rule: A successful deployment means software was placed and verified in an environment. It does not mean a proposed cloud state is trusted. Product trust still requires the normal authorization, impact, DAIL verification, governance, promotion, and authoritative confirmation path.